

IT 362 Course Project
Semester-2, 1447H

Project Logbook – Phase 3

Prepared by

Noora Aluqaili	445201450
Jana Alothman	445200267
Haya Alhajri	445202280
Noura Altuwiam	445202179
Rahaf Alshalwi	445201250

Section : 66849

Supervised by : Dr.Reem Alqifari

1. Dataset Preparation for Modeling

What was done:

We loaded the cleaned dataset prepared in Phase 2 and verified its structure and columns.

Why:

To ensure that the dataset is complete and ready for modeling.

How:

- Used `pd.read_csv()`
- Checked dataset shape and columns

Decision & Rationale:

We relied on the cleaned dataset from Phase 2 to avoid repeating preprocessing and ensure consistency across all models.

Result:

Dataset contains **1034 records and structured features** ready for modeling.

1. Creating Sentiment Labels

What was done:

We converted numerical ratings into sentiment categories.

Why:

Classification models require categorical labels instead of raw numerical ratings.

How:

- 1–2 → negative
- 3 → neutral
- 4–5 → positive

Decision & Rationale:

We grouped ratings this way because it reflects real user sentiment (negative, neutral, positive) and simplifies the classification task.

Result:

New column **sentiment** created with three classes.

3. Checking Sentiment Distribution

What was done:

We analyzed the distribution of sentiment classes.

Why:

To understand dataset balance before training.

How:

- Used `value_counts()`

Decision & Rationale:

We checked distribution early to decide whether special handling (like stratification or different metrics) is needed.

Result:

- Positive: 527
- Negative: 444
- Neutral: 63

Dataset is imbalanced, especially for neutral class.

4. Additional Arabic Text Preprocessing

What was done:

We applied extra cleaning to Arabic text.

Why:

To improve text quality and reduce noise before modeling.

How:

- Removed non-Arabic characters
- Normalized letters
- Cleaned spacing
- Created `model_text`

Decision & Rationale:

We added this step even after Phase 2 cleaning because machine learning models are sensitive to text inconsistency, especially in Arabic.

Result:

Text became more uniform and suitable for feature extraction.

5. Handling Missing Values

What was done:

Removed rows with missing or empty text.

Why:

Empty text cannot contribute to model learning.

How:

- Used `dropna()`
- Filtered rows with `length > 0`

Decision & Rationale:

Keeping empty values would introduce noise and reduce model accuracy.

Result:

Clean dataset maintained for training.

6. Splitting Data

What was done:

Split data into training and testing sets.

Why:

To evaluate model performance on unseen data.

How:

- 80/20 split
- Used `train_test_split()` with `stratify`

Decision & Rationale:

We used stratified splitting to preserve class distribution due to dataset imbalance.

Result:

- Training: 827
- Testing: 207

7. Feature Extraction (TF-IDF)

What was done:

Converted text into numerical features using TF-IDF.

Why:

Models cannot process raw text directly.

How:

- Used `TfidfVectorizer(max_features=5000)`

Decision & Rationale:

TF-IDF was chosen instead of simple frequency methods because it gives more importance to meaningful words and reduces the impact of common words.

Result:

Text transformed into numerical vectors ready for modeling.

8. Model 1: Logistic Regression (Baseline)

What was done:

Trained Logistic Regression model.

Why:

To create a baseline for comparison.

How:

- Trained on TF-IDF features
- Predicted on test set

Decision & Rationale:

Logistic Regression is simple, fast, and commonly used as a baseline to evaluate improvements from more complex models.

Result:

- Accuracy: 0.78
- Macro F1-score: 0.54

Model performed well on positive/negative but failed on neutral class.

9. Model 2: Random Forest

What was done:

Trained Random Forest model.

Why:

To capture non-linear patterns in the data.

How:

- Used `RandomForestClassifier(n_estimators=100)`

Decision & Rationale:

Random Forest was selected to test whether ensemble learning improves performance over the baseline.

Result:

- Accuracy: 0.71
- Macro F1-score: 0.48

Performance decreased, especially for neutral class.

10. Model 3: XGBoost

What was done:

Trained XGBoost model.

Why:

To use a more advanced boosting technique.

How:

- Encoded labels using `LabelEncoder`
- Used `XGBClassifier`
- Converted predictions back

Decision & Rationale:

XGBoost was chosen because it is known for high performance and handling complex patterns efficiently.

Result:

- Accuracy: 0.71
- Macro F1-score: 0.52. Slight improvement over Random Forest but still below baseline

11. Model Evaluation Approach

What was done:

Evaluated models using multiple metrics.

Why:

Accuracy alone is not enough for imbalanced datasets.

How:

- Used Accuracy
- Used Macro F1-score

Decision & Rationale:

Macro F1-score was selected as the main metric because it treats all classes equally, especially important since the neutral class is underrepresented.

Result:

Provided fair comparison across models.

12. Model Comparison

What was done:

Compared all models.

Why:

To identify the best-performing model.

How:

Compared Accuracy and Macro F1.

Result:

- Logistic Regression → best performance
- XGBoost → moderate
- Random Forest → lowest

12. Model Comparison

What was done:

We conducted a structured comparison between all three models developed in this phase: Logistic Regression, Random Forest, and XGBoost. The comparison was based on their performance on the same testing dataset.

Why: This step is important to identify which model performs best and is most suitable for the sentiment classification task.

How: We compared the models using two main evaluation metrics: Accuracy and Macro F1-score. These metrics were calculated after evaluating each model on the test data. The results were then organized side-by-side for clear comparison.

Decision & Rationale:

We chose to rely on both Accuracy and Macro F1-score because Accuracy alone may be misleading when dealing with imbalanced data. Macro F1-score gives equal importance to all classes (positive, negative, neutral), making it more reliable for fair evaluation.

Result:

The comparison showed clear differences in model performance:

- **Logistic Regression** achieved the best overall performance, with higher and more balanced results across classes.
- **XGBoost** showed moderate performance, performing well on major classes but still struggling with the minority class.
- **Random Forest** had the lowest performance and did not significantly improve classification results.

Overall, Logistic Regression was selected as the best-performing model for this task.

13. Challenges

- Imbalanced dataset (few neutral samples)
- Difficulty predicting neutral class
- Arabic text complexity

14. Solutions

- Used stratified sampling
- Applied additional text preprocessing
- Used Macro F1-score for fair evaluation

15. Final Status

- Dataset fully prepared
- Three models trained and evaluated
- Results saved for final comparison

13. Confusion Matrix Analysis

What was done:

We generated confusion matrices for all three models (Logistic Regression, Random Forest, and XGBoost) using the test dataset.

Why:

To gain deeper insight into model performance by analyzing how predictions are distributed across actual classes, especially for imbalanced data.

How:

- Used `confusion_matrix` from `sklearn.metrics`
- Visualized results using heatmaps for each model
- Compared true labels vs predicted labels

Decision & Rationale:

We used confusion matrices because evaluation metrics like accuracy and F1-score do not show detailed class-level errors. Confusion matrices help identify which classes are misclassified, particularly the minority class (neutral).

Result:

The confusion matrices showed that all models performed well in predicting positive and negative classes. However, all models struggled with the neutral class, often misclassifying it as positive or negative. Logistic Regression showed the most balanced performance, while Random Forest and XGBoost had more misclassifications. This confirms that class imbalance negatively affected the prediction of the neutral class.